

浙江大学 实验报告

姓名：马玉玺
学号：3240104676
日期：2025 年 7 月 7 日

实验名称：

Vectorization

一、AVX-512 实验思路

我们的目标是利用 512 位宽的 AVX 寄存器，一次并行计算出结果矩阵 C 的一行中的 16 个元素，实现计算 $C = A * B$ 。

算法 0 的整个计算过程可以分解为以下几个步骤：

1. 外层循环 (i):略
2. 中层循环 (j):

```
for (int j = 0; j < N; j += 16)
```

这个循环以 16 为步长，遍历矩阵 B 和 C 的列。选择步长为 16 的原因是，一个 512 位宽的 AVX 寄存器 (`__m512i`) 刚好可以容纳 16 个 32 位整数 ($512 / 32 = 16$)。因此，这个循环的每一次迭代，都是为了计算出 $C[i][j]$ 到 $C[i][j+15]$ 这 16 个元素的结果。

3. 初始化累加器:

```
__m512i c_vec = _mm512_setzero_si512();
```

在开始计算这 16 个元素之前，先初始化一个 512 位宽的向量寄存器 `c_vec`，并将其所有位都设为 0。这个向量将用作累加器，累加存储 16 个点积的中间结果。

4. 内层循环 (k) - 核心计算:

```
for (int k = 0; k < K; k++)
```

这个循环是执行点积累加操作的核心。在每次迭代中广播 A 的元素：

```
__m512i a_broadcast_vec = _mm512_set1_epi32(*(uint32_t*)&A[i * K * 4 + k * 4]);
```

从矩阵 A 中取出 4 个 `uint8_t` (即一个 `uint32_t`)，然后使用 `_mm512_set1_epi32` 指令将这个 32 位数值复制 16 次，填满整个 512 位宽的向量寄存器 `a_broadcast_vec`。加载 B 的元素：

```
__m512i b_vec = _mm512_loadu_si512((__m512i const*)&B_reshape[k * N * 4 + j * 4]);
```

从预先重排过的 `B_reshape` 矩阵中，加载一段连续的 512 位 (64 字节) 数据到向量寄存器 `b_vec`。对 B 进行重排是为了让后续的内存访问更加高效。

点积累加：

```
c_vec = _mm512_dpbusd_epi32(c_vec, a_broadcast_vec, b_vec);
```

这是最关键的一步，`_mm512_dpbusd_epi32` 是一条复合指令，它会将 `a_broadcast_vec` 和 `b_vec` 这两个 512 位向量中的字节进行点积运算，再将结果累加到 `c_vec` 向量的对应位置。

5. 存储结果：

```
_mm512_storeu_si512((__m512i*)&C[i * N + j]), c_vec);
```

当内层 `k` 循环结束后，`c_vec` 中就保存了 `C[i][j]` 到 `C[i][j+15]` 这 16 个元素的最终计算结果。此指令将这个向量寄存器的内容一次性写回到 `C` 矩阵的内存中。

二、AVX-512 算法调整

以下是我的两种算法调整思路：

算法 1:

```
for (int i = 0; i < M; i++) {
    for (int j = 0; j < N; j += 32) { // 循环展开，每次处理32列
        __m512i c_vec0 = _mm512_setzero_si512();
        __m512i c_vec1 = _mm512_setzero_si512
        for (int k = 0; k < K; k++) {
            __m512i a_broadcast_vec = _mm512_set1_epi(uint32_t*)&A[i * K * 4 +
k * 4]);
            __m512i b_vec0 = _mm512_loadu_si512((__m512i constB_reshape[k * N
* 4 + j * 4]));
            __m512i b_vec1 = _mm512_loadu_si512((__m512i constB_reshape[k * N
* 4 + (j + 16) * 4]
            c_vec0 = _mm512_dpbusd_epi32(c_vec0, a_broadcast_vb_vec0);
            c_vec1 = _mm512_dpbusd_epi32(c_vec1, a_broadcast_vb_vec1);
        }
        _mm512_storeu_si512((__m512i*)&C[i * N + j]), c_vec0);
        _mm512_storeu_si512((__m512i*)&C[i * N + j + 16], c_vec1);
    }
}
```

算法 1 的优势主要体现在以下三个方面：

并行度提升:使用两个向量寄存器同时处理 32 列数据,相比 16 列实现了更高的指令级并行。

内存访问优化:通过交错加载 B 矩阵的两个块,内存访问间隔为 64 字节(恰为一个缓存行),减少了缓存冲突。

寄存器利用效率:双寄存器策略降低了寄存器压力,延长了数据重用时间,减少了循环内的依赖链。

算法 2:

```

for (int i = 0; i < M; i++) {
    for (int j = 0; j < N; j += 16) {
        __m512i c_vec = _mm512_setzero_si512();
        for (int k_block = 0; k_block < K; k_block += 4) {
            for (int k = 0; k < 4 && k_block + k < K; k++) {
                int k_idx = k_block + k;

                if (k_block + k + 4 < K) {
                    _mm_prefetch(&B_reshape[(k_idx+4) * N * 4 + j * 4],
_MM_HINT_T0);
                }

                __m512i a_broadcast = _mm512_set1_epi32(*(uint32_t*)&A[i * K *
4 + k_idx * 4]);

                __m512i b_vec = _mm512_loadu_si512(
                    (__m512i const*)&B_reshape[k_idx * N * 4 + j * 4]
                );

                c_vec = _mm512_dpbusd_epi32(c_vec, a_broadcast, b_vec);
            }
        }

        if (j + 16 > N) {
            uint32_t* c_ptr = &C[i * N + j];
            uint32_t* vec_ptr = (uint32_t*)&c_vec;

            for (int l = 0; l < N - j; l++) {
                c_ptr[l] = vec_ptr[l];
            }
        } else {
            _mm512_storeu_si512((__m512i*)&C[i * N + j], c_vec);
        }
    }
}

```

算法 2 实际上会比算法 0 和算法 1 都要慢,它的劣势主要体现在以下两个方面:

预取指令开销:预取指令本身消耗 CPU 周期,在数据局部性好或 K 值较小时,收益不足以抵消开销。

代码复杂度代价:额外的安全检查和分块逻辑增加了指令数量,降低了计算密度。

三、 Bonus:AMX 实验思路

AMX 操作主要分为两个部分:Tile 模块的配置与初始化和利用 Tile 执行矩阵乘法

3.1. AMX Tile 模块配置与初始化

这部分代码在第一个 start 和 end 注释块之间。它的作用是配置 AMX 的 Tile 模块,告诉 CPU 我们打算如何使用 Tile 寄存器(tmm0-tmm7)。Tile 是 AMX 引入的二维寄存器,用于加速矩阵运算。

3.2. 利用 AMX Tile 执行矩阵乘法

这部分代码在第二个 start 和 end 注释块之间,是实际执行矩阵乘法的核心。它利用了“外积”的思想,通过分块(tiling)的方式,逐步计算出最终结果。

核心指令: `tile_dpbusrd(0, 2, 3)`

这是 AMX 的核心计算指令。它执行 tmm2 和 tmm3 中两个 Tile 的外积(tile outer product)。dpbusrd 表示它执行的是 int8 和 uint8 字节(Byte)的点积(Dot Product),并将结果(int32)累加(Sum)到目标 Tile 中。0 表示目标累加器是 tmm0。

我的理解:放弃逐个元素计算的模式,改为以“块”Tile 为单位进行运算。核心循环就是三步:用 `_tile_loadr` 将 A 和 B 的一小块数据“搬”进配置好的 Tile 寄存器,然后用一条指令 `_tile_dpbusrd` 完成这两个小块矩阵的乘加运算,最后用 `_tile_store` 将累加好的结果块“搬”回内存。一言以蔽之:AMX 的思路就是将传统的、软件控制的循环计算,转变为由硬件(AMX 单元)直接执行的、高效的矩阵块(Tile)运算,从而实现数量级的性能提升。

三、附件

```
h3240104676@sct101:~/hpc101/src/lab2/vector/build$ srun -p M7 ./lab2
Time: 0.112212 s
Time: 0.0672931 s
Result is correct!
Speedup: 1.66751
```

Figure 1: AVX 算法 0 正确性、加速比和运行时间

```
h3240104676@sct101:~/hpc101/src/lab2/vector/build$ srun -p M7 ./lab2
Time: 0.199471 s
Time: 0.0490824 s
Result is correct!
Speedup: 4.064
```

Figure 2: AVX 算法 1 正确性、加速比和运行时间

```
h3240104676@sct101:~/hpc101/src/lab2/vector/build$ srun -p M7 ./lab2
Time: 0.112193 s
Time: 0.109178 s
Result is correct!
Speedup: 1.02762
```

Figure 3: AVX 算法 2 正确性、加速比和运行时间

```
h3240104676@sct101:~/hpc101/src/lab2/vector/build$ srun -p M7 ./lab2
Time: 2.97525 s
Time: 0.0247423 s
Result is correct!
Speedup: 120.25
```

Figure 4: AMX 正确性、加速比和运行时间

```
vector — gaia@MacBook-Air — ..c/lab2/vector — -zsh — -zsh — 123x37
-- Detecting C compile features — done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info — done
-- Check for working CXX compiler: /usr/bin/c++ — skipped
-- Detecting CXX compile features
-- Detecting CXX compile features — done
-- Configuring done
-- Generating done
-- Build files have been written to: /src/build
[ 25%] Building CXX object CMakeFiles/lab2.dir/main.cpp.o
[ 50%] Building CXX object CMakeFiles/lab2.dir/src/buffer.cpp.o
[ 75%] Building CXX object CMakeFiles/lab2.dir/src/reshape.cpp.o
[100%] Linking CXX executable lab2
[100%] Built target lab2
exit code: 0
2025-07-07 09:54:03.543 running workflow 1 step 3 / 3
2025-07-07 09:54:03.642 running workflow 2 / 3
2025-07-07 09:54:03.828 running workflow 2 step 1 / 2
2025-07-07 09:54:06.935 running workflow 2 step 2 / 2
$ cd $SOJ_REAL_WORKDIR && su soj-user -c "srun --time 00:00:10 -p M7 --ntasks=1 --cpus-per-task=1 -N 1 ./lab2"
start
init_buffer done
naive_gemm done
gemm done
result is correct
time: 69ms
exit code: 0
2025-07-07 09:54:07.347 running workflow 3 / 3
2025-07-07 09:54:07.523 running workflow 3 step 1 / 1
2025-07-07 09:54:11.245 Submission completed
Submit is completed
Message:
    judge successfully finished
Score 74.86 max.100 (Unweighted)
Judgement Message:
    Correct, time: 69ms
```

Figure 5: AVX 算法 00J 运行结果

```
vector — gaia@MacBook-Air — ..c/lab2/vector — -zsh — -zsh — 80x24
[100%] Built target lab2
exit code: 0
2025-07-10 16:58:50.377 running workflow 1 step 3 / 3
2025-07-10 16:58:50.473 running workflow 2 / 3
2025-07-10 16:58:50.671 running workflow 2 step 1 / 2
2025-07-10 16:58:53.780 running workflow 2 step 2 / 2
$ cd $SOJ_REAL_WORKDIR && su soj-user -c "srun --time 00:00:10 -p M7 --ntasks=1 --cpus-per-task=1 -N 1 ./lab2"
start
init_buffer done
naive_gemm done
gemm done
result is correct
time: 51ms
exit code: 0
2025-07-10 16:58:54.191 running workflow 3 / 3
2025-07-10 16:58:54.403 running workflow 3 step 1 / 1
2025-07-10 16:58:58.148 Submission completed
Submit is completed
Message:
    judge successfully finished
Score 85.14 max.100 (Unweighted)
Judgement Message:
    Correct, time: 51ms
```

Figure 6: AVX 算法 10J 运行结果

```
vector — gaia@MacBook-Air — ../lab2/vector — -zsh - -zsh — 80x24

[100%] Built target lab2
exit code: 0
2025-07-10 17:00:01.841 running workflow 1 step 3 / 3
2025-07-10 17:00:01.944 running workflow 2 / 3
2025-07-10 17:00:02.129 running workflow 2 step 1 / 2
2025-07-10 17:00:05.215 running workflow 2 step 2 / 2
$ cd $SOJ_REAL_WORKDIR && su soj-user -c "srun --time 00:00:10 -p M7 --n
tasks=1 --cpus-per-task=1 -N 1 ./lab2"
start
init_buffer done
naive_gemm done
gemm done
result is correct
time: 114ms
exit code: 0
2025-07-10 17:00:05.658 running workflow 3 / 3
2025-07-10 17:00:05.863 running workflow 3 step 1 / 1
2025-07-10 17:00:09.562 Submission completed
Submit is completed
Message:
    judge successfully finished
Score 49.14 max.100 (Unweighted)
Judgement Message:
    Correct, time: 114ms
```

Figure 7: AVX 算法 2OJ 运行结果

```
vector — gaia@MacBook-Air — ../lab2/vector — -zsh - -zsh — 123x37

-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /src/build
[ 25%] Building CXX object CMakeFiles/lab2.dir/main.cpp.o
[ 50%] Building CXX object CMakeFiles/lab2.dir/src/buffer.cpp.o
[ 75%] Building CXX object CMakeFiles/lab2.dir/src/reshape.cpp.o
[100%] Linking CXX executable lab2
[100%] Built target lab2
exit code: 0
2025-07-07 09:52:44.871 running workflow 1 step 3 / 3
2025-07-07 09:52:44.974 running workflow 2 / 3
2025-07-07 09:52:45.189 running workflow 2 step 1 / 2
2025-07-07 09:52:48.281 running workflow 2 step 2 / 2
$ cd $SOJ_REAL_WORKDIR && su soj-user -c "srun --time 00:00:10 -p M7 --ntasks=1 --cpus-per-task=1 -N 1 ./lab2"
start
init_buffer done
naive_gemm done
gemm done
result is correct
time: 19ms
exit code: 0
2025-07-07 09:52:48.639 running workflow 3 / 3
2025-07-07 09:52:48.841 running workflow 3 step 1 / 1
2025-07-07 09:52:52.584 Submission completed
Submit is completed
Message:
    judge successfully finished
Score 100.00 max.100 (Unweighted)
Judgement Message:
    Correct, time: 19ms
```

Figure 8: AMXOJ 运行结果

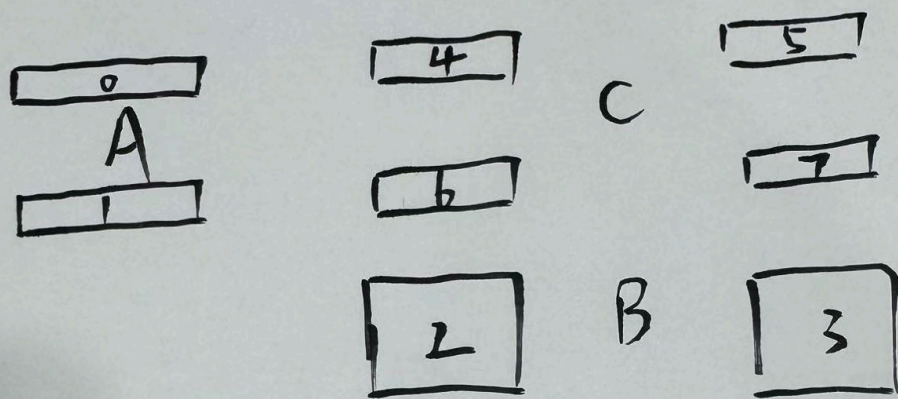


Figure 9: Bonus